

AI TRANSIT PIPELINE

Securing AI Code Integration in the Enterprise

- Secure retrieval from GitHub
- 5-layer adaptive multi-layer scan by file type
- Approved ZIP archive + Excel traceability report

Table of Contents

01 Context & Objectives	02 Pipeline Architecture — 5 Layers	03 Phase 1 — Secure Retrieval
04 Layer 1 — Global Scan	05 Layer 2 — OWASP Top 10 / CWE Top 25	06 Layer 3 — SCA: Vulnerable Dependencies
07 Layer 4 — Universal Patterns	08 Layer 5 — Per-type SAST (11 languages)	09 Tools Used
10 Results & Deliverables	11 Absolute Security Rules	

01 Context & Objectives

Why this pipeline?

- AI-generated code may contain backdoors, secrets, or dangerous patterns
- Developers import code without systematic security review
- The corporate network must remain isolated (partial air-gap)
- No traceability without a structured process



Key Objectives

Retrieve

Secure depth-1 clone

Scan

5-layer adaptive pipeline

Decide

PASS→ZIP / FAIL→Quarantine

Trace

Excel + JSON + HTML

02 Pipeline Architecture — 5-Layer Scanning

Top-level Flow

■ Source
GitHub / Local

■ fetch_repo.sh
(Phase 1)

■ scan_pipeline.sh
(Phase 2 — 5 Layers)

PASS ▼

FAIL ▼

✓ Good/ — ZIP + Excel

✗ quarantine/ chmod 700

5-Layer Scanning Cascade



PASS

FAIL

03 Phase 1 — Secure Retrieval

1 Host Whitelist

Only github.com is allowed.

Any other URL is rejected immediately.

2 Size Verification

GitHub API queried before clone.

Limit: 500 MB.

3 Minimal Clone

```
git clone --depth 1 --no-tags  
  
--single-branch
```

4 .git/ Removal

Git metadata removed

(hooks, remotes, submodules).

5 SHA-256 Manifest

Hash of each file →

.manifest_sha256.txt (audit trail).

6 Quick Triage

Grep: eval(exec(curl|bash

rm -rf → immediate alert.

04 Layer 1 — Global Scan (all files)

AV · gitleaks · detect-secrets · YARA — Applies to the ENTIRE directory, regardless of file type

■ Gitleaks

- > 150 credential types
- GitHub/AWS/GCP/Azure tokens
- RSA, PEM, SSH, JWT keys
- Entropy analysis + regex

FAIL if positive

■ detect-secrets

- Shannon entropy
- Secrets unknown to rule sets
- High-density base64/hex
- Complements Gitleaks

FAIL if positive

■ ClamAV / AV

- Millions of signatures
- Trojans, ransomware, backdoors
- Full recursive scan
- freshclam updated base

FAIL if positive

■ YARA

- Industry-standard IOC
- Custom .yar rules
- AI backdoor patterns
- Regex + boolean logic

FAIL if positive

05 Layer 2 — OWASP Top 10 2021 + CWE Top 25 (Semgrep)

4 Semgrep rulesets run against all code files

p/owasp-top-ten	p/cwe-top-25	p/security-audit	p/secrets
OWASP Top 10 2021 - A01 — Broken Access Control - A02 — Cryptographic Failures - A03 — Injection (SQLi, XSS, SSTI...) - A04 — Insecure Design - A05 — Security Misconfiguration - A06 — Vulnerable Components - A07 — Auth Failures - A08 — Data Integrity Failures - A09 — Logging & Monitoring Failures - A10 — SSRF	CWE Top 25 Most Dangerous - CWE-79 XSS - CWE-89 SQL Injection - CWE-20 Improper Input Validation - CWE-125 Out-of-bounds Read - CWE-78 OS Command Injection - CWE-416 Use After Free - CWE-22 Path Traversal - CWE-352 CSRF - CWE-476 NULL Pointer Dereference - CWE-287 Improper Authentication	General Security Audit - Broad vulnerability patterns - Misconfigurations - Insecure API usage - Deprecated functions - Crypto weaknesses - Race conditions - Integer overflows - Memory safety issues - Protocol misuse - Hardcoded values	Secrets & Credentials - API keys (AWS, GCP, Azure...) - Private keys (RSA, EC, PEM) - OAuth tokens - Database connection strings - JWT secrets - Slack / Twilio / Stripe tokens - Generic password patterns - High-entropy strings - Environment variable leaks - Config file secrets

06 Layer 3 — SCA: Vulnerable Dependencies (OWASP A06)

What is SCA?

Software Composition Analysis (SCA) identifies known vulnerabilities (CVEs) in third-party libraries and dependencies.

OWASP A06:2021 — Vulnerable and Outdated Components is consistently in the OWASP Top 10.

■ trivy

Universal SCA scanner

- All ecosystems (pip, npm, gem, go, cargo...)
- CVE database (NVD, OSV, GitHub Advisory)
- Scans requirements.txt, package-lock.json, go.sum, Gemfile.lock, Cargo.lock...
- CRITICAL / HIGH / MEDIUM / LOW severity
- Output: JSON + table report
- Also scans Dockerfiles & container images
- Regularly updated vulnerability DB

FAIL on CRITICAL/HIGH

■ pip-audit / safety

Python — requirements*.txt

- pip-audit: official PyPA tool (PURL-based)
- safety: PyUp.io database
- Scans requirements.txt, requirements-dev.txt
- requirements-*.txt (glob pattern)
- Maps to CVE + GHSA identifiers
- Detects yanked / abandoned packages
- Outputs FAIL on any CRITICAL finding
- Complements trivy for Python ecosystem

FAIL on CRITICAL/HIGH

■ npm audit

JavaScript / Node — package-lock.json

- Built-in npm command (no install needed)
- Uses npm registry advisory database
- Scans package-lock.json
- Severity: critical / high / moderate / low
- FAIL on critical or high findings
- Reports affected package tree
- Identifies fix version when available
- Covers direct and transitive dependencies

FAIL on CRITICAL/HIGH

07 Layer 4 — Universal Patterns (all files)

Pattern-matched against EVERY file regardless of language — no tool dependency

CWE / Standard	Pattern / Rule	Example	Verdict
CWE-798	Hardcoded credentials	password="...", api_key="..."	FAIL
CWE-321	Hardcoded crypto key	BEGIN RSA PRIVATE KEY	FAIL
CWE-22	Path traversal	../../etc/passwd	FAIL
CWE-918	SSRF	requests.get(url=user_input)	WARN
CWE-327	Weak cryptography	md5, sha1, rc4	WARN
CWE-338	Weak PRNG	Math.random(), rand()	WARN
OWASP-A09	Logging suppressed	logging.disable(logging.CRITICAL)	WARN

08 Layer 5 — Per-type SAST Dispatch (11 languages)

Extension(s)	SAST Tool	Manual Checks	Verdict
.py	Bandit	eval() exec() import os/subprocess	FAIL
.js .ts .jsx	Semgrep	eval() child_process require(...)	FAIL
.java	Semgrep	Runtime.exec() ObjectInputStream JNDI Log4Shell TrustAllCerts	FAIL
.php	Semgrep	shell_exec() echo \$_GET include(\$var) unserialize()	FAIL
.rb	Semgrep	system/exec eval() ActiveRecord string interpolation	FAIL
.go	Semgrep	fmt.Sprintf in SQL math/rand TLS MinVersion	FAIL
.c .cpp .h	cppcheck	gets() strcpy() system() popen()	FAIL
.sh .bash .ps1	ShellCheck	curl bash wget sh eval \$var	FAIL
.yml .yaml	—	Unpinned actions inline secrets	FAIL
.tf .tfvars	checkov	Hardcoded secrets overly broad IAM	FAIL
Dockerfile	hadolint	:latest ADD http:// RUN curl bash	FAIL
.xml	—	DOCTYPE + ENTITY (XXE CWE-611)	FAIL
.so .exe .elf	strings	Auto FAIL + IOC strings	FAIL ■
.zip .tar.gz	—	Auto FAIL — re-scan after extraction	FAIL ■
.sql	—	xp_cmdshell DROP TABLE ; --	FAIL

08 Layer 5 — Per-type Check Details

Python .py

Dynamic execution — high risk of backdoors via eval/exec

Bandit SAST

FAIL

Python AST: subprocess shell=True, pickle, MD5, SQL, assert...

eval() / exec()

FAIL

Arbitrary execution at runtime — trivial backdoor vector

import os/subprocess

WARN

Shell access — requires manual review

JavaScript .js .ts

Ecosystem exposed to supply-chain attacks

Semgrep p/javascript

FAIL

XSS, prototype pollution, innerHTML, SSTI...

eval()

FAIL

Dynamic execution — classic injection vector

child_process

FAIL

Shell command execution from Node.js

08 Layer 5 — Per-type Check Details

C / C++ .c .cpp

Risks from manual memory management

cppcheck

FAIL

Buffer overflows, null ptr, use-after-free, uninit memory.

gets() strcpy()

FAIL

Unbounded read/copy → buffer overflow

system() popen()

FAIL

Shell command execution → possible injection

Shell .sh .bash

Direct execution — command obfuscation is trivial

ShellCheck

FAIL

Unquoted variables, globbing, dangerous redirections...

curl | bash

FAIL

Remote code execution without integrity check

eval \$variable

FAIL

Trivial injection if variable controlled by attacker

08 Layer 5 — Per-type Check Details

YAML / CI .yml

CI/CD pipelines — a bad config opens backdoors

Unpinned actions

FAIL

uses: action@main → vulnerable to tag-hijacking

Inline secrets

FAIL

password: value (not \${ secrets.XXX })

Terraform .tf

IaC — can provision entire cloud resources

checkov (CIS)

FAIL

S3 buckets, overly broad IAM, DB without at-rest encryption

Hardcoded secrets

FAIL

password = "value" in .tf/.tfvars

08 Layer 5 — New Languages: Java · PHP · Ruby · Go · XML

Java .java

Runtime reflection & deserialization — critical attack surface

Runtime.exec() (CWE-78)

FAIL

OS command injection via Java runtime shell execution

ObjectInputStream (CWE-502)

FAIL

Deserialization of untrusted data — RCE vector

Log4Shell JNDI (CVE-2021-44228)

FAIL

JNDI lookup in log messages → critical 0-day pattern

TrustAllCerts (CWE-295)

FAIL

Disabled TLS certificate validation → MITM

PHP .php

Direct user input to shell / output — classic web attack surface

shell_exec() (CWE-78)

FAIL

Direct shell execution with user-controlled input

echo \$_GET (CWE-79)

FAIL

Reflected XSS — user input echoed without sanitization

include(\$var) (CWE-22)

FAIL

Remote/local file inclusion via variable path

unserialize() (CWE-502)

FAIL

PHP deserialization → object injection & RCE

08 Layer 5 — New Languages: Java · PHP · Ruby · Go · XML

Ruby .rb

Metaprogramming power creates significant injection risk

system/exec (CWE-78)

FAIL

Shell command injection via system() or exec()

eval() (CWE-95)

FAIL

Dynamic code evaluation — trivial code injection

ActiveRecord interpolation (CWE-89)

FAIL

SQL injection via string interpolation in queries

Go .go

Low-level control — subtle SQL and crypto pitfalls

fmt.Sprintf in SQL (CWE-89)

FAIL

String-formatted SQL queries → SQL injection

math/rand (CWE-338)

WARN

Weak PRNG — not cryptographically secure

TLS MinVersion (CWE-326)

FAIL

Weak TLS version allowed (< TLS 1.2)

XML .xml

XML External Entity (XXE) — file read & SSRF via XML parsers

DOCTYPE + ENTITY (CWE-611)

FAIL

XXE: reads local files or makes internal HTTP requests

SYSTEM entity

FAIL

<!ENTITY xxe SYSTEM 'file:///etc/passwd'>

08 Layer 5 — Dockerfile · Binaries · Archives · SQL

Dockerfile

• **hadolint**

FAIL

CIS Docker: :latest, ADD http://, apt without --no-install-recommends

• **RUN curl|bash**

FAIL

Download + execution without integrity control

Binaries .so .exe .elf

• **Auto FAIL**

FAIL

No binaries in an AI repo — potential implant/RAT

• **strings + IOC**

FAIL

/bin/sh /etc/passwd exec reverse shell URLs detected

Archives .zip .tar.gz

• **Auto FAIL**

FAIL

Content cannot be scanned without prior extraction

• **Zip-slip**

FAIL

Extraction to arbitrary paths (../../etc/cron.d/)

SQL .sql

• **xp_cmdshell**

FAIL

Shell command execution from SQL Server → critical IOC

• **DROP TABLE / --**

FAIL

Destructive statements + SQL injection pattern

09 Tools Used — Summary

All optional in degraded mode — WARN if absent, FAIL only on active finding

Tool	Role	Installation	Scope
Gitleaks	secrets/tokens	GitHub binary	L1 Global
detect-secrets	high entropy	pip install	L1 Global
ClamAV	malware signatures	apt install clamav	L1 Global
YARA	custom IOC	apt install yara	L1 Global
Semgrep	OWASP/CWE rulesets (×4)	pip install semgrep	L2 OWASP/CWE
trivy	SCA universal CVE scan	GitHub binary	L3 SCA
pip-audit	Python SCA (PyPA)	pip install pip-audit	L3 SCA
safety	Python SCA (PyUp.io)	pip install safety	L3 SCA
npm audit	Node.js SCA (built-in)	built-in npm	L3 SCA
Bandit	Python SAST	pip install bandit	L5 .py
cppcheck	C/C++ static analysis	apt install cppcheck	L5 .c .cpp
ShellCheck	shell linting	apt install shellcheck	L5 .sh
hadolint	Dockerfile linting	GitHub binary	L5 Docker
checkov	IaC security	pip install checkov	L5 .tf
jq	GitHub API JSON parsing	apt install jq	Utility

10 Results — ZIP Archive & Excel Report

■ ZIP Archive → Good/

```
Good/
  repo_20260615_143022.zip
  src/
    [full source code]
  README.md
  scan_report_20260615.xlsx
```

- ✓ Produced only if verdict is PASS
- ✓ Timestamped name — guaranteed uniqueness
- ✓ .manifest_sha256.txt excluded
- ✓ Excel report included directly
- ✓ Ready for internal network transfer
- ✓ Optional: GPG encryption

■ Excel Report (.xlsx)

Tab 0 — Summary

Repository / Source	GitHub URL or local path
Scan Date	ISO 8601 timestamp
SHA-256 Hash	Global fingerprint of all files
Verdict	PASS (green) / FAIL (red)
Counters	PASS · WARN · FAIL

Tab 1 — Files (one row per file)

#	File	Type	Status	Message
1	src/main.py	.py	FAIL	bandit:HIGH
2	config.sh	.sh	WARN	shellcheck absent
3	README.md	.md	PASS	—

11 Absolute Security Rules

■ These rules CANNOT be modified — they constitute the threat model

■ Network Isolation

The transit machine NEVER has direct access to the corporate network.

➔ One-way Flow

Only approved/ is readable from the inside — not fetch/, logs/, quarantine/.

■ Root-only Quarantine

chmod 700 — no application user can read rejected files.

■ No Manual Move

Never quarantine/ → approved/ without a full pipeline re-scan.

■ Binaries = Absolute FAIL

No .so/.exe/.elf in an AI repo — zero exceptions.

■ Archives = Mandatory Re-scan

.zip/.tar.gz → extraction + re-scan in a dedicated isolated environment.

Summary



5-layer pipeline · OWASP · CWE · CERT · SCA · Degraded mode · Timestamped reports